Notebook Construction and Audit Strategies

This section outlines two complementary frameworks for working with computational notebooks in machine learning and data science contexts: a *Construction Framework* for authoring new notebooks, and an *Audit Framework* for formally reviewing completed drafts. While iterative evaluation is a natural part of notebook construction, applying a formal audit too early can disrupt the exploratory flow. Therefore, these frameworks are applied sequentially within a conditional feedback loop, illustrated in Figure 1, where audit findings drive subsequent construction revisions.

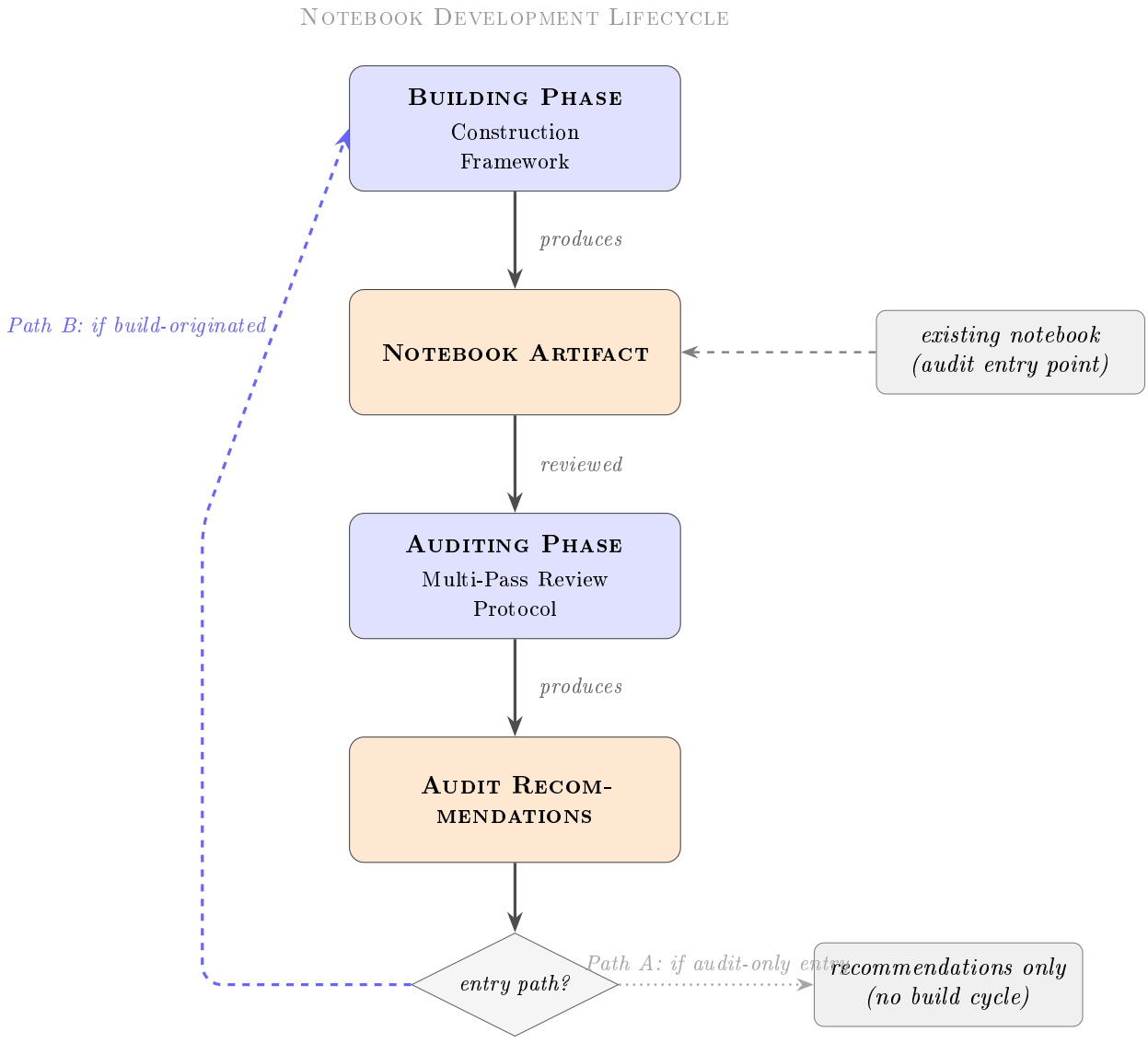

Figure 1: The iterative lifecycle of a computational notebook. Initial drafting and exploratory evaluation occur under the Construction Framework (Part I) before transitioning to the formal Audit Framework (Part II). Recommendations then resolve through one of two mutually exclusive paths depending on entry point: Path B closes the loop back into the Building Phase for notebooks that originated there; Path A terminates at the recommendations for notebooks that entered directly via the audit-only route, unless subsequently routed into construction.

Part I — Notebook Construction Framework

The Construction Framework is a *forward-looking prescription*: it guides the notebook author in producing a well-structured, reproducible, and maintainable artifact from the ground up. It is organized into three sequential phases.

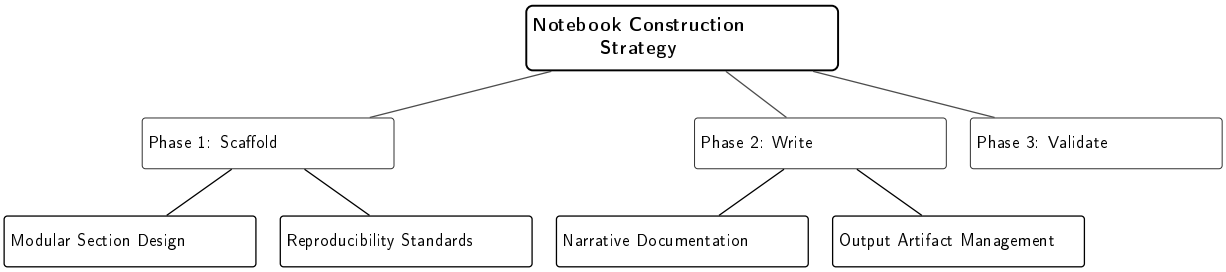

Figure 2: High-level pillars of the Notebook Construction Framework.

Phase 1 — Scaffold: Establish the structural and reproducibility foundation before writing any logic:

- Define the notebook's **single responsibility** — one notebook should correspond to one coherent workflow or experiment.
- Create standard section headers as markdown cells, following this canonical order:
  1. Environment & Dependencies
  2. Configuration & Global Parameters
  3. Data Ingestion & Schema Validation
  4. Data Partitioning & Split Manifest
  5. Preprocessing & Feature Engineering
  6. Model Definition & Training
  7. Evaluation & Metrics
  8. Artifact Export (models, plots, reports)
  9. Conclusions & Next Steps

  Data Partitioning and Split-Manifest Contract When data partitioning is applicable, the *Data Partitioning & Split Manifest* section shall define the split strategy, split keys, random or chronological boundaries, grouping constraints, stratification rules, and the immutable assignment of observations or groups to the required partitions before any fitted preprocessing, feature selection, model fitting, threshold selection, hyperparameter selection, or model selection is performed.

  The Split Manifest shall be bound to the immutable identity of the exact source-data snapshot from which the partition assignment was generated. At minimum, it shall record:

  – the dataset name and immutable version, release identifier, registry identifier, or equivalent snapshot reference;

- the integrity reference for every local input file or for the versioned dataset manifest governing a collection of input files;
- the applicable schema or data-contract version;
- the expected observation or group identifier set, represented directly or through a verified integrity reference;
- the stable observation or group identifier used for partitioning;
- the partition assigned to each observation or group, either directly or through a versioned assignment artifact;
- the partitioning method and its relevant configuration;
- the random seeds, temporal boundaries, grouping keys, stratification variables, filtering rules, and exclusion criteria used to generate the assignment; and
- the integrity reference for the completed Split Manifest and any linked assignment artifact.

A split assignment shall be valid only for the exact dataset snapshot, schema, observation set, filtering and deduplication rules, and partitioning configuration recorded in the Split Manifest. Any material change to those elements shall invalidate the previous Split Manifest and require generation of a new manifest before preprocessing, training, or evaluation continues.

Before the split is consumed, the implementation shall verify that the observed dataset identity matches the recorded source-data identity, that each expected observation or group is assigned consistently, that no prohibited overlap exists across partitions, and that all declared grouping, chronological, and stratification constraints are satisfied.

Phase 2 — Write: Author each section incrementally, following these discipline rules:

- Work through **one section at a time** (Divide and Conquer principle).
- Accompany every code cell with a markdown explanation of intent and expected output.
- Avoid burying logic inside loops or helper calls without commentary.
- Prefer **explicit variable passing** over hidden or global state to maintain cell independence.
- Summarize repetitive code patterns with a general description and highlight meaningful variations. Three or more similar blocks shall be treated as a refactoring candidate. Extracted logic may be moved into a local Python module only when that module is versioned, included in the project repository, listed in the Audit Package Manifest, and reviewed together with the notebook.
- Route all persisted outputs (models, plots, reports) through a single, versioned export convention defined at the start of the section — never write artifacts ad hoc from arbitrary cells.

Phase 3 — Validate During Writing: Continuously verify correctness as each section is completed while maintaining a strict separation between partial-development executions and complete release-bearing runs.

Execution-Attempt and Release-Run Contract Every checkpoint, exploratory, or partial execution shall receive a unique *Execution Attempt Identifier*. Its execution record shall identify:

- the governing manifest versions;
- the notebook section or workflow stage executed;
- the execution status;
- the artifacts produced; and
- whether the execution was partial or complete.

Artifacts produced by a partial execution shall be stored in a development namespace associated with the Execution Attempt Identifier. They shall not be written to, promoted into, or interpreted as artifacts of a complete release run.

A release-bearing *Run Identifier* shall be used only for a fresh-kernel, end-to-end execution of the complete applicable workflow. Its lifecycle shall use the following states:

`COMPLETE_UNVERIFIED` and `COMPLETE_VERIFIED`.

A complete run shall remain `COMPLETE_UNVERIFIED` until all required section contracts, data constraints, expected outputs, and artifact-integrity checks have been verified. Only then may it transition to `COMPLETE_VERIFIED` and support release, publication, or deployment.

- After finalizing a section, restart the kernel and execute all cells up to that section under a new Execution Attempt Identifier.
- Verify that the section consumes only declared inputs, produces the documented outputs and side effects, and does not depend on execution of later cells or undeclared mutable state.
- Evaluate section-level reproducibility using the applicable deterministic checks or predefined stochastic tolerances.
- Store all checkpoint outputs under the corresponding Execution Attempt Identifier. Partial execution shall never overwrite, replace, or verify a complete-run artifact.
- Resolve execution failures, manifest mismatches, undeclared state, partition inconsistencies, or missing expected outputs before proceeding.
- After all applicable sections are complete, restart the kernel and execute the notebook end-to-end under the release Run Identifier.
- Verify the complete artifact set before marking the run `COMPLETE_VERIFIED`.
- Re-execution shall create a new execution-attempt record. It may verify an existing release artifact set when the governing run identity is unchanged, but it shall not silently overwrite verified release artifacts.
- Any change to a governing input that changes run identity shall produce a distinct release run and artifact set.

Part II — Notebook Audit Framework

The Audit Framework is a *backward-looking diagnostic*: it guides a reviewer, collaborator, or AI assistant in systematically assessing the quality, correctness, and risk profile of an existing notebook.

Audit Package Boundary Before execution, an *Audit Package Manifest* shall enumerate all supplied files, their project-relative paths, and their cryptographic hashes. The audit target shall comprise the complete executable project surface required by the notebook:

- the `.ipynb` file;
- every transitively imported project-local module;
- project configuration and schema files;
- environment and dependency manifests;
- referenced local data contracts or split manifests; and
- scripts used to execute, export, or validate the notebook.

Third-party package source code is outside the audit boundary unless explicitly included, but its package name and resolved version remain part of the reproducibility review.

If required project-local code or configuration is absent, Level 1 shall record a Critical reproducibility finding. Methodological or deployment logic delegated to a supplied local module shall be audited in the same level as equivalent inline notebook logic.

To execute this efficiently, the reviewer should divide the notebook into logical modules, prioritize critical pipeline blocks over repetitive exploration cells, use hierarchical descriptions, and summarize repeated code patterns. Before execution, every audit step shall be classified as `IN_SCOPE`, `OUT_OF_SCOPE`, or `NOT_APPLICABLE`. A user exclusion produces `OUT_OF_SCOPE`; a step is `NOT_APPLICABLE` only when the notebook type does not contain the corresponding operation, and the reviewer records objective evidence for that determination.

Passes 2–6 shall execute every `IN_SCOPE` step. Neither `OUT_OF_SCOPE` nor `NOT_APPLICABLE` contributes to a risk score. A final unrestricted pass requires completion of every applicable step with no user exclusions. Any audit containing an `OUT_OF_SCOPE` step is a limited-scope audit and shall not receive an unrestricted release status.

The framework is formalized as a six-pass review protocol, applied sequentially. Any pass included in scope is executed according to its applicability classification and deliverable requirements below.

Finding Severity and Deterministic Risk Assignment Every finding shall be assigned exactly one severity:

Critical: The finding prevents required execution, invalidates scientific results, exposes credentials or sensitive data, or independently blocks release, deployment, or publication.

Major: The finding materially weakens reproducibility, methodological correctness, maintainability, or deployment readiness but is not independently release-blocking.

Minor: The finding affects clarity, documentation, output hygiene, or non-blocking maintainability without changing execution or results.

Each executed level shall receive an aggregate risk score based on all verified findings assigned to that level:

- *Low Risk*: no Critical findings, no Major findings, and no more than two Minor findings.
- *Moderate Risk*: no Critical findings and either one or two Major findings or three or more Minor findings.
- *High Risk*: one or more Critical findings or three or more Major findings within the level.

Across the complete audit, four or more verified Major findings shall escalate the aggregate audit risk to *High Risk*, even when no single level contains three Major findings. A finding shall not affect a final risk score until its evidence and source reference have been verified. LLM-generated findings and pass claims remain provisional until verification is complete.

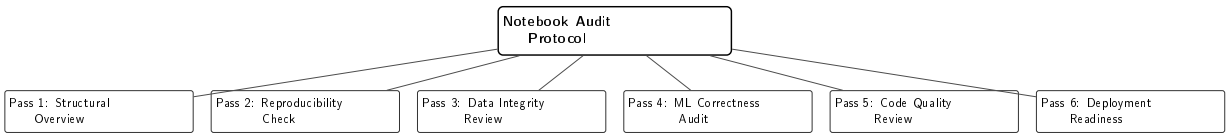

Figure 3: High-level pillars of the Notebook Audit Framework.

Pass 1 — Structural Overview: Establish a high-level map of the notebook before examining any logic:

- Record any user-specified focus areas or exclusions that will scope the remaining five passes.
- Assess whether the notebook adheres to the canonical section headers defined in the Construction Framework (e.g., Environment, Configuration, Ingestion).
- Confirm that the notebook has a single, coherent purpose.
- Verify that cell execution order is linear and safe.
- Identify execution-relevant structural red flags (e.g., broken imports, missing required cells, unsafe execution order, or unresolved project-local dependencies). Code smells and unused elements remain exclusive to Pass 5.
- Perform the mandatory safety pre-screen as defined in the Verification Workflow (item 1). This pre-screen is never excludable and completes before any other stop condition is evaluated.
- *Deliverable:* Section map, structural findings, mandatory safety pre-screen, and Audit Scope Manifest.

Pass 2 — Reproducibility Check: Assess whether the notebook can be reliably re-executed:

- Verify that dependencies are declared and version-pinned.
- Confirm that stochastic controls are configured for the tech stack (e.g., global random seeds and framework-specific deterministic flags for deep learning workloads).
- Flag any hardcoded file paths, credentials, or environment-specific assumptions.
- After the mandatory Pass 1 safety pre-screen has completed and no unresolved safety block remains, execute the notebook end-to-end from a fresh kernel in an isolated environment constructed from the declared dependency manifest. Record the command, environment hash, exit status, execution log, runtime, and produced artifact manifest. A verified Low or Moderate reproducibility score shall not be issued without this execution evidence.
- *Deliverable:* Clean-execution record and aggregate Level 1 reproducibility risk score (Low / Moderate / High), derived from structural, dependency, execution, and safety findings.
- *Deterministic Gate:* A *High Risk* score on this deliverable triggers a HARD STOP. Passes 3–6 must not be executed unless an explicit override was declared during Pass

1's scope recording.

Pass 3 — Data Integrity Review: Examine the data pipeline for correctness and leakage risks:

- Confirm that no information from the test or validation set leaks into the training process (e.g., ensure imputers, scalers, encoders, or feature selection algorithms are fitted only on the training data).
- Verify that missing data is handled consistently across splits.
- Check schema, data types, shapes, ranges, and invariant constraints at ingestion. Distributional profiling used to guide model or preprocessing decisions shall be restricted to the training partition; test-partition checks shall be limited to predefined contract validation and final evaluation.
- *Deliverable:* Data pipeline integrity report.
- *Deterministic Gate:* Verified pipeline-level data leakage invalidates the affected model-performance values, comparisons, selection decisions, and conclusions. Pass 4 design checks remain reportable as independent `PASS` or `FLAG` findings.

Pass 4 — ML Correctness Audit: Evaluate the soundness of the machine learning methodology:

- Confirm that the evaluation metric is appropriate for the task and class distribution. Verify that cross-validation is applied correctly and without leakage.
- Ensure hyperparameter selection uses only training-derived resampling, cross-validation, or a designated validation partition. Nested cross-validation is valid when the outer assessment folds remain isolated. The final test set shall never guide tuning or model selection.
- Check that model performance is contextualized against a meaningful baseline.
- *Deliverable:* ML correctness checklist stamped `PASS` or `FLAG`, accompanied by a separate performance-claim validity status when leakage is verified.

Pass 5 — Code Quality Review: Assess the maintainability and clarity of the code:

- Flag repetitive code blocks that exceed the three-block threshold as refactoring candidates.
- Identify dead code, unused imports, and redundant variable assignments.
- Assess whether variable and function names are meaningful and consistent.
- Check for bloated cell outputs (e.g., printing entire dataframes or raw tensors).
- *Deliverable:* Code quality and smell report whose verified findings contribute to the aggregate Level 3 risk score.

Pass 6 — Deployment Readiness: Determine whether the notebook is suitable for production or publication:

- Verify compliance with the *Execution-Attempt and Release-Run Contract* defined in Part I: checkpoint and partial outputs shall remain associated with Execution Attempt Identifiers in the development namespace, while a release-bearing Run Identifier shall apply only to a fresh-kernel, end-to-end execution of the complete applicable workflow.

- Confirm that release artifacts remain `COMPLETE_UNVERIFIED` until the required contracts, constraints, outputs, and integrity checks have been verified, and that only `COMPLETE_VERIFIED` artifacts are eligible for publication, deployment, or release.
- Confirm that re-execution creates a new execution-attempt record and does not silently overwrite, replace, or verify release artifacts with outputs from a partial execution.
- Confirm that inference logic is separable from training logic.
- Check that compute and memory constraints are documented or profiled.
- Ensure a complete environment export is present and up to date.
- Flag any data privacy or PII handling concerns.
- *Deliverable:* Deployment readiness/risk score (Low / Moderate / High).

LLM Execution Assumptions and Verification Protocol

The prompt architecture in the Prompt Crafting section delegates complex analytical tasks to Large Language Models. To deploy this framework safely, the following assumptions about LLM execution must be acknowledged and mitigated:

- **The Triage Sensor Model:** LLM outputs are high-recall, lower-precision triage reports. The LLM is highly effective at scanning for structural patterns (e.g., a missing seed, a `fit_transform` on test data), but it cannot be trusted to perfectly trace complex dynamic state or deeply nested logic.
- **Context Window Constraints:** The chained 3-Level architecture (Lap 3) mitigates but does not eliminate context degradation. If a notebook exceeds the effective attention span of the target LLM (typically less that 600 lines of dense code for strict formatting adherence), the prompts must not be run monolithically. The notebook must be chunked programmatically, or the LLM must be given tool-use access (e.g., Python REPL) to execute cells and inspect state directly.
- **Mandatory Human-in-the-Loop (HITL):** Every LLM status is provisional. Before any gate or risk score is finalized, a human reviewer or approved deterministic tool MUST verify both positive findings and pass claims against recorded evidence. Absence of an LLM flag is not evidence of compliance.

Authority Boundary:

An LLM may generate provisional findings, provisional severities, and provisional gate recommendations. It shall not issue a final `PASS`, `FAILED`, `HARD STOP`, or `INVALIDATED` decision.

After each LLM level, the workflow shall enter `PAUSED_FOR_VERIFICATION` with Gate State `PAUSE`, regardless of whether the LLM reported a flag. A human reviewer or an approved deterministic analysis tool shall verify the flags, pass claims, applicability decisions, and evidence before any transition to `PROCEED` or `STOP`.

Only verified outcomes may be converted into final severities, risk scores, invalidations, and gate decisions in the Final Audit Manifest.

Approved Execution Runner and Containment Contract An LLM shall not independently claim that it built an environment, executed project code, or verified runtime behavior. It may request execution only through a human-authorized or approved deterministic runner operating under the containment requirements defined below.

The approved runner shall return an evidence record containing:

- the exact execution command or workflow invocation;
- the audited source and package identity;
- the resolved environment identity;
- the containment configuration applied;
- the execution start and completion states;
- the exit status;
- references to standard-output and standard-error logs;
- the observed runtime and resource-limit outcomes; and
- the produced artifact manifest.

Notebook execution shall occur within an isolated, non-privileged environment with the following default controls:

- project source and declared input data mounted read-only;
- a dedicated writable output location restricted to the execution;
- no access to host credentials, user secrets, environment secrets, or unrelated filesystem locations;
- network access denied by default and enabled only through an explicitly recorded authorization or allowlist;
- process, runtime, memory, storage, and other applicable resource limits;
- no privileged operations, host-level configuration changes, or writes outside the authorized execution surface;
- no destructive modification of source data, project files, prior artifacts, or external resources; and
- controlled substitutes, recorded snapshots, or explicit authorization for required external services.

Environment construction and notebook execution shall be treated as separate controlled stages. The execution stage shall not install, upgrade, or replace dependencies unless that action was explicitly authorized and recorded as part of the resolved environment.

If safe containment cannot be established, or execution requires unapproved network access, credentials, privileged operations, unrestricted external side effects, or writes outside the authorized surface, execution shall not proceed. The audit shall enter `PAUSED_FOR_VERIFICATION` with Gate State `PAUSE`, or remain `AUDIT INCOMPLETE` when the required execution capability is unavailable rather than unsafe.

LLM interpretations of runner evidence remain provisional until the evidence, control outcomes, and resulting findings have been verified.

Verification Workflow:

1. **Mandatory non-executing safety pre-screen (authoritative definition; referenced from Pass 1 and Step 1.2):** Before building an environment, importing project code, or executing any notebook cell, inspect the notebook, supplied local modules, configuration

files, stored outputs, and export paths for exposed credentials, secrets, directly displayed sensitive records, unsafe persistence, or code paths that may transmit or disclose protected information.

The safety pre-screen shall be ground-truthed before clean execution begins. If a credible exposure or execution-time disclosure risk is identified, set Level Status to `PAUSED_FOR_VERIFICATION` and Gate State to `PAUSE`. Do not begin the clean-environment run until the finding is rejected or contained and execution is explicitly authorized. The pre-screen result remains in the Audit Manifest even when the audit terminates at Level 1.

2. **Context window compliance check:** Before accepting any Lap 3 level's output as eligible for verification, confirm the notebook's reported line count against the declared execution mode. If the notebook exceeds the effective attention-span threshold stated in the Context Window Constraints assumption and the level's self-report declares monolithic execution, reject that level's output as `NON_COMPLIANT_EXECUTION`, discard its findings, and re-run the level in chunked or tool-use mode before any further verification proceeds. Record the compliance outcome in the Audit Manifest regardless of result.

3. **Triage and execution:** After the mandatory safety pre-screen has cleared, complete the remaining Level 1 controls, including the clean-environment run. Execute later levels only when the Gate State is `PROCEED`, or as explicitly authorized supplementary reviews after a verified failure.

4. **De-duplication:** Aggregate validly generated findings and replace duplicates with references to their original finding identifiers.

5. **Evidence verification:** Verify every provisional flag, every severity assignment, every `NOT_APPLICABLE` classification, and every pass claim. A pass claim requires affirmative evidence for the control being assessed.

6. **Adjudication:** Record confirmed, rejected, and unresolved outcomes in the Audit Manifest. Final risk and gate decisions shall be issued only after all in-scope and applicable controls have reached a terminal verification state.

Applying the Two Frameworks

The appropriate framework depends on the context of engagement with the notebook:

Use the Construction Framework when: starting a new notebook or experiment from a blank state; or creating a reusable notebook template for a team or project.

Use the Audit Framework when: reviewing a collaborator's completed notebook; preparing a notebook for production deployment or academic publication; conducting a pre-merge review; or using an AI assistant to analyze submitted notebook code without intending to rewrite it.

Use Both Frameworks iteratively when: authoring and self-reviewing during active development (applying Construction Phases 1–3, supplemented by targeted Audit checks as sections are completed); collaborating or pair-programming in real time; or refactoring an existing notebook into a cleaner, production-ready version, regardless of whether its original structure is preserved or rebuilt.

The Construction Framework and Audit Framework complement each other within a conditional feedback loop: the Construction phase produces a notebook artifact, the Audit phase diagnoses its weaknesses, and the resulting recommendations feed back into a revised construction cycle. If

a notebook enters the workflow strictly through the Audit Framework (e.g., reviewing a peer's work), the loop is paused unless the reviewer transitions to active construction to fix the identified issues, as illustrated in Figure 1.

Stable Source-Reference Protocol

All audit laps and levels shall use a common source-reference map tied to the audited source hash.

For each notebook cell:

1. Use the native notebook `cell.id` value when present.
2. If no native identifier exists, assign a fallback identifier according to the cell's zero-based physical position in the audited snapshot, formatted as `C000`, `C001`, and so forth.
3. Preserve identifiers for markdown, raw, empty, failed, and executable cells. Excluding a cell from a report shall not renumber the remaining cells.

Display block numbers may be added for readability, but they are not authoritative references. Every display block shall list its underlying source identifiers, for example:

```
Block 4 [Source cells:  C007, C009]
```

A grouped Environment Setup block shall list every source cell included in the group. References to imported local modules shall use the project path and line range, for example:

```
src/preprocessing.py:45-78
```

All findings, conflicts, verification records, and revisions shall use these stable source references.

Prompt Crafting

The following prompts document the evolution of the framework. Lap 1 and Lap 2 are archival prototypes for exploratory comparison only. They do not implement the Audit Scope Manifest, applicability states, clean-execution evidence, finding records, verification workflow, or inter-level gate contract and therefore shall not be used for formal, release-bearing, publication-bearing, or regulatory audits. Only Lap 3, with the contracts defined in this proposal, is an operational audit procedure.

Lap 1. Archival coarse evaluation (non-gating)

TASK: Analyze the provided Jupyter Notebook and produce a structured audit. Proceed to systematically analyze the codebase, examining each section sequentially.

PHASE 1 — EXTRACTION & DOCUMENTATION Execute the following four steps in order: - Output "Section 1: Overall Purpose" and provide a 2-3 sentence summary.

- Output "Section 2: Code Block Descriptions". Assign display block numbers in top-to-bottom order, but preserve and report the stable source-cell identifier for every executable cell. Do not renumber source cells after filtering or grouping. For each block, extract and list: Logic and Flow, Data and State, Operations and Methods, and Side Effects.

- Output "Section 3: Complex Logic Translation". Identify any functions that are too complex to parse at a glance. Provide plain-English pseudocode for these specific functions only.

- Output "Section 4: Pipeline Summary". Write 3-5 sentences explaining block dependencies, if they exist.

PHASE 2 — CROSS-REFERENCE CHECK Re-scan the original notebook code. Compare the code's actual behavior against the documentation you generated in Phase 1. Point out any discrepancies between what the code actually does and what you documented.

PHASE 3 — CRITICAL AUDIT Evaluate and score the ORIGINAL CODE (ignore your Phase 3 pseudocode) for the following:

- Cross-block consistency in data types and variable naming

- Redundant computation or unnecessary I/O

- Contradictory logic or unreachable branches

- Missing error handling or validation

PHASE 4 — TRIAGE SUMMARY Summarize the observed risks without assigning a gate, release status, formal risk score, or verification state. This archival prompt shall not be used as an operational audit.

append fingerprint "lap 1", prompt version, and current UTC timestamp

---

Lap 2. Archival monolithic evaluation (non-gating)

Task: Analyze the provided Jupyter Notebook and its supplied local project modules. Assign display block numbers sequentially according to top-to-bottom visual appearance, independently of Jupyter execution counters. For every display block, include the stable source-cell identifier or identifiers defined by the Stable Source-Reference Protocol. Display block numbers shall not replace source references.

1 - Overall Purpose / Goal. Provide a clear and concise summary (2-3 sentences maximum) of the notebook's main objective, including:

- The problem it solves

- The type of input it expects

- The output or result it produces

2 - Conceptual Description of Each Code Block. Apply the following rules for processing cells:

- Consolidation: Group all library imports, package installations (e.g., !pip), and magic commands into a single first block titled "Environment Setup".

- Filtering: Skip pure markdown cells, raw text cells, and genuinely empty cells from the executable block description. Do not skip a code cell because it raises an exception, has a missing output, or was not successfully executed. Include such a cell as a separate block, record its stable source identifier, describe the attempted operation, and report the observed or evident failure under Side Effects.

For all remaining valid executable code blocks, extract:

- Logic Flow: Step-by-step reasoning behind the code's execution.

- Data Transformations: What goes in (inputs) and what comes out (outputs) regarding data shapes/types (e.g., "Transforms DataFrame X from wide to long format").

- Model/Algorithm Operations: Any ML, statistical, or computational methods applied.

- Key Variables/Parameters: Important identifiers, hyperparameters, or configurations.

- Side Effects: File I/O, state changes, printed output, API calls, etc.

3 - Pipeline Summary. Explain how the extracted blocks connect to form a complete pipeline:

- Trace the data flow from the first computational block to the last.

- Detail any branching, looping, or conditional paths across blocks.

4 - Cross-Block Code Quality Audit. Audit the notebook's underlying code for structural and logical issues. Address each of the following points strictly using bullet points:

- Coherence and Consistency: [Note any mismatches between data transformations and downstream usage]

- Redundancy and Waste: [Identify any inefficient repeated computation or redundant data transformations]

- Contradictory Logic: [Point out any instances where later blocks logically contradict or overwrite earlier blocks]

If no issues are found in a category, write "None detected."

OUTPUT FORMAT REQUIREMENTS. Structure your response exactly as follows, using the exact headings provided:

1 - Overall Purpose / Goal [Your summary here]

2 - Code Block Descriptions

Block 1 - Environment Setup [Source cells: Cxxx, Cyyy]. Logic Flow: N/A Data Transformations: N/A Model/Algorithm Operations: N/A Key Variables/Parameters: [List imported libraries/versions if specified] Side Effects: [List any installations or magic command outputs]

Block 2 - [Short Descriptive Title] [Source cells: Cxxx, Cyyy]. Logic Flow: ... Data Transformations: ... Model/Algorithm Operations: ... Key Variables/Parameters: ... Side Effects: ... [Continue pattern for all blocks...]

3 - Pipeline Summary. [Your pipeline explanation here]

4 - Cross-Block Code Quality Audit. [Your bulleted audit here]

append fingerprint "lap 2" and current timestamp

Lap 3. Modular 3-Level evaluation

While Lap 2 relies on a single monolithic prompt, Lap 3 represents an architectural alternative motivated by established LLM limitations— specifically, the "lost-in-the-middle" phenomenon where models ignore instructions buried in long contexts, and the degradation of strict formatting adherence as token counts rise. Lap 3 maps the six audit passes onto a 3-level chained prompting architecture to mitigate these constraints by strictly isolating the LLM's attention scope per level:

- *Level 1 — Conceptual* (Passes 1–2): Structural Overview + Reproducibility Check. Establishes whether the notebook is coherent and re-runnable at all, before any deep inspection is worth doing.
- *Level 2 — Methodological* (Passes 3–4): Data Integrity Review + ML Correctness Audit. Addresses whether the pipeline and modeling logic are scientifically sound — the "does this work correctly" layer.
- *Level 3 — Implementation* (Passes 5–6): Code Quality Review + Deployment Readiness. Addresses how the working logic is written and packaged — the most granular, code-level layer.

A Note on Framework Efficacy and Meta-Baselining: Just as Step 2.8 of the Audit Framework mandates that model performance must be contextualized against a baseline, the architectural claims of this prompt-engineering proposal require empirical validation. The transition from the monolithic Lap 2 prompt to the chained Lap 3 architecture is theoretically motivated by LLM attention constraints, but it remains an unverified hypothesis regarding audit efficacy (e.g., false positive reduction, leakage detection recall, or formatting compliance). Rigorous adoption of this framework should include ablation studies comparing Lap 2 and Lap 3 against a ground-truth dataset of flawed notebooks to establish a quantifiable baseline for the prompts themselves.

Audit Scope and Applicability Contract Level 1 shall create an *Audit Scope Manifest* before substantive analysis begins. For every level and step, the manifest shall record one of the following states:

`IN_SCOPE`: the step is applicable and shall be executed;

`OUT_OF_SCOPE`: the user explicitly excluded the step, with the exclusion rationale and request identifier recorded; or

`NOT_APPLICABLE`: the notebook does not contain the operation governed by the step, supported by objective evidence.

The mandatory Level 1 safety pre-screen cannot be excluded or marked `NOT_APPLICABLE`. The manifest shall also record focus areas, predeclared supplementary-review overrides, and whether the audit is `FULL_SCOPE` or `LIMITED_SCOPE`.

The Audit Scope Manifest shall be passed unchanged to every subsequent level. A later level shall not broaden, narrow, or reinterpret scope or applicability without a recorded reviewer-authorized amendment. A `NOT_APPLICABLE` classification shall be verified like any other pass claim. Any `OUT_OF_SCOPE` step forces a limited-scope final status and shall never be represented as a pass.

Level 1 – Conceptual Purpose Determine whether the notebook is coherent and reliably re-executable before investing time in deeper review.

Step 1.1 — Scope the audit: Record any user-specified focus areas or exclusions (if a collaborator has scoped the review in advance). Note this scope explicitly — it determines which of the remaining passes/levels get full attention later.

Step 1.2 — Perform the mandatory non-executing safety pre-screen: Execute the mandatory non-executing safety pre-screen as defined in the Verification Workflow (item 1). This step is never excludable and completes before any other stop condition is evaluated, including Step 1.6 (clean execution).

Step 1.3 — Map the structure: Assess whether the notebook has clear, navigable section headers. Confirm the notebook has a single, coherent purpose (not multiple unrelated experiments bundled together). Verify cell execution order is linear and free of hidden dependencies.

Step 1.4 — Scan for execution-relevant structural failures: Identify broken imports, missing required cells, unresolved local modules, unsafe execution order, and other structural conditions that can prevent or alter end-to-end execution. Do not assess unused imports, orphaned analysis code, naming, or other Pass 5 concerns.

Step 1.5 — Check reproducibility inputs: Verify dependencies are declared and version-pinned (requirements.txt, conda.yaml, pyproject.toml, or equivalent). Confirm that stochastic controls are configured for the tech stack: global random seeds at minimum, and framework-specific deterministic flags (e.g., PyTorch, TensorFlow, JAX) if deep learning workloads are present.

Step 1.6 — Execute the reproducibility test:

Only after Step 1.2 has completed, no unresolved safety block remains, and the approved execution runner has confirmed the required containment controls, submit the complete notebook to the runner for fresh-kernel, end-to-end execution.

The runner shall construct or select the resolved environment, execute the notebook without manual intervention, and return the evidence record required by the Approved Execution Runner and Containment Contract.

The reviewer or LLM shall evaluate only the returned runner evidence and shall not claim execution that is not supported by that record.

If execution cannot be attempted because the audit package, environment, runner, or required contained resource is unavailable, the audit remains `AUDIT INCOMPLETE`. If execution is withheld because a containment or safety condition is unresolved, Level Status shall be `PAUSED_FOR_VERIFICATION` and Gate State shall be `PAUSE`. If contained execution is attempted and fails, the verified cause contributes to the Level 1 risk score.

Level 1 deliverables: Section map, structural findings, Audit Scope Manifest, clean-execution record, mandatory safety pre-screen, aggregate Level 1 risk score (Low / Moderate / High), manifest delta, and provisional or verified Gate State.

Deterministic Gate Decision: A *High Risk* reproducibility score triggers a **HARD STOP**. In that case, Levels 2 and 3 shall not be executed, and the audit terminates after Level 1.

The only exception is an explicit override recorded during Step 1.1 before the audit begins, such as:

> *Evaluate methodological or implementation risks despite unresolved reproducibility failures.*

When such an override exists, Levels 2 and 3 may be executed only as supplementary diagnostic reviews. Their findings shall not alter the Level 1 failure status or support approval of the notebook.

Corresponding LLM Prompt:

You are performing a Level 1 (Conceptual) audit of a Jupyter notebook. You are acting as a high-recall triage sensor for a Jupyter notebook. Process the provided code within your context window. Your role is to identify probable risks, not to render a definitive legal or scientific verdict. Because LLMs are subject to attention limits and hallucination, your outputs MUST be treated as hypotheses requiring human verification, not established facts.

Your ONLY objective at this level is to determine whether the notebook is structurally coherent and reliably re-executable. Do NOT evaluate code quality, ML methodology, data leakage, or deployment readiness — those belong to later audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — SCOPE AND APPLICABILITY - Create the Audit Scope Manifest. Classify every step as IN_SCOPE, OUT_OF_SCOPE, or NOT_APPLICABLE and record objective evidence for every NOT_APPLICABLE classification. - The mandatory safety pre-screen is always IN_SCOPE. - Mark the audit FULL_SCOPE only when no step is OUT_OF_SCOPE.

STEP 2 — MANDATORY NON-EXECUTING SAFETY PRE-SCREEN - Before building an environment, importing project code, or executing any notebook cell, inspect the notebook, supplied local modules, configuration

files, stored outputs, and export paths for exposed credentials, API keys, secrets, directly displayed sensitive records, unsafe persistence, or code paths that may transmit or disclose protected information. - This step is never excludable. - If a credible exposure or execution-time disclosure risk is identified, set Level Status to PAUSED_FOR_VERIFICATION and Gate State to PAUSE. Do not perform clean execution until the finding is rejected or contained and execution is explicitly authorized.

STEP 3 — STRUCTURE MAP - List the notebook's section headers (markdown cells) in order. - State whether the notebook has a single, coherent purpose, or if it bundles multiple unrelated workflows/experiments. - Verify cell execution order is linear — flag any cell that appears to depend on a later cell, or any out-of-order execution counts visible in cell outputs.

STEP 4 — STRUCTURAL EXECUTION SCAN - Identify broken imports, missing required cells, unresolved local modules, unsafe execution order, and cells whose non-execution prevents the declared workflow from completing. - Do not report unused imports, orphaned analysis code, naming, or other code-quality findings at this level. - Resolve every project-local import against the Audit Package Manifest and cite stable references for every failure.

STEP 5 — REPRODUCIBILITY INPUTS - Check whether stochastic controls are configured for the tech stack. Look for global random seeds (numpy, python, random). - If the notebook uses deep learning frameworks (PyTorch, TensorFlow, JAX), check for framework-specific deterministic flags. - Flag if a notebook performs stochastic GPU operations but relies solely on basic python/numpy seeds without configuring hardware-level determinism.

STEP 6 — CLEAN EXECUTION - Proceed only after Step 2 has completed, no unresolved safety block remains, and the approved execution runner has confirmed the required containment controls. - Submit the complete notebook and declared execution inputs to the approved runner for fresh-kernel, end-to-end execution. - Do not claim that you built the environment or executed the notebook. Evaluate only the runner evidence returned under the Approved Execution Runner and Containment Contract. - Record the resolved environment identity, containment configuration, command, exit status, execution-log references, runtime, resource-limit outcomes, and artifact manifest. - Write "NOT EXECUTED — RUNNER UNAVAILABLE" when the required execution capability is unavailable and "NOT EXECUTED — SAFETY HOLD" when safety or containment authorization is unresolved. - Do not recommend a verified Low or Moderate risk score without successful contained-execution evidence.

OUTPUT FORMAT — return exactly these seven items, followed by the machine-readable Manifest Delta, and nothing else:

1. Audit Scope Manifest: step state, rationale, request identifier, applicability evidence, overrides, and FULL_SCOPE or LIMITED_SCOPE.

2. Mandatory non-executing safety pre-screen: provisional status, evidence, Level Status, Gate State, and explicit clean-execution authorization or prohibition.

3. Section map and structural execution findings, with evidence and stable source references.

4. Clean-execution record: environment hash, command, exit status, log reference, runtime, and artifact-manifest reference; write "NOT EXECUTED — SAFETY HOLD" when Step 2 prohibits execution.

5. Provisional Level 1 risk score: Low / Moderate / High, derived from Steps 2–6 under the deterministic severity rubric.

6. Provisional state and post-verification recommendation: - Current Level Status: PAUSED_FOR_VERIFICATION. - Current Gate State: PAUSE. - Recommend PROCEED after verification only when the verified Level 1 risk is Low or Moderate. - Recommend STOP after verification when a release-blocking Level 1 finding is confirmed. A supplementary-review override shall be recorded but shall not erase a verified STOP state.

7. Evidence-backed control outcomes: for each executed control, emit PROVISIONAL_PASS, PROVISIONAL_FLAG, OUT_OF_SCOPE, or NOT_APPLICABLE. A provisional pass requires affirmative evidence.

Manifest Delta: append all required manifest fields and structured finding records, including unique IDs, evidence, provisional severity, verification state, affected claims, Level Status, and Gate State.

Do not add fixes. Do not comment on code style, naming, or model correctness.

Level 2 – Methodological Purpose Determine whether the applicable data pipeline and modeling methodology are scientifically sound after Level 1 has completed. Level 2 may run in normal mode when the preceding Gate State is `PROCEED`, or in supplementary mode only when a predeclared override authorizes review after a verified Level 1 failure. Supplementary findings cannot change the earlier failure status.

Step 2.1 — Verify information flow: Locate the point at which observations are assigned to training, validation, and test partitions. Confirm that this assignment occurs before any fitted, target-informed, or data-adaptive transformation.

Schema inspection, deterministic parsing, immutable identifier creation, and explicitly declared label-independent filtering may occur before the split. These operations shall not be flagged unless their parameters or selection rules are estimated from the complete dataset.

Flag a structural flow violation only when transformation parameters, category vocabularies, feature-selection criteria, imputation values, normalization statistics, sampling decisions, or comparable data-dependent rules are learned before partition isolation.

Step 2.2 — Check for pipeline-level leakage: Identify specific algorithmic instances of leakage (e.g., scalers, encoders, imputers, or dimensionality reduction algorithms) explicitly fitted on the full dataset rather than the training split alone.

Step 2.3 — Check missing-data handling: Verify that missing data is handled consistently across the train and test splits (same strategy applied to both, fit only on training data).

Step 2.4 — Validate ingestion contracts without exposing the test distribution: Validate schema, data types, shapes, ranges, and invariant constraints at load time. Restrict exploratory or decision-guiding distributional profiling to the training partition. Apply only predefined contract checks to validation and test partitions before final evaluation.

Step 2.5 — Verify cross-validation integrity: Confirm cross-validation folds preserve the train/test separation established in Step 2.1. Check that no fold-selection, resampling, or preprocessing-within-CV step exposes validation or test data to the training process.

Step 2.6 — Check evaluation metric appropriateness: Confirm the evaluation metric fits the task and class distribution (e.g., accuracy on an imbalanced classification problem is a red flag).

Step 2.7 — Check hyperparameter-selection boundaries: Ensure hyperparameter and model selection use only training-derived resampling, cross-validation, or a designated validation partition. Nested cross-validation is valid when outer assessment folds remain isolated. The final test set shall not guide selection.

Step 2.8 — Check baseline comparison: Confirm model performance is contextualized against a meaningful baseline (e.g., a naive predictor, a simpler model, or a published benchmark) — not reported in isolation.

Level 2 deliverables: Data pipeline integrity report, ML correctness checklist, performance-claim validity status, aggregate Level 2 risk score, Gate State transition, and manifest delta. Every applicable pass and every `NOT_APPLICABLE` classification requires verified evidence.

Deterministic Gate Decision: A provisional Critical methodological finding sets Level Status to `PAUSED_FOR_VERIFICATION` and Gate State to `PAUSE`. If pipeline-level leakage or another Critical methodological failure is verified, Level Status becomes `TERMINATED` and Gate State becomes `STOP`; affected performance claims are invalidated. Level 3 shall not execute afterward unless the Audit Scope Manifest contains a predeclared supplementary implementation-review override. If no Critical finding is verified but three or more Major findings are verified within Level 2, the aggregate Level 2 risk score is High Risk under the Finding Severity rubric; Level Status becomes `TERMINATED` and Gate State becomes `STOP` on the same basis as a Critical finding, and Level 3 shall not execute afterward unless the Audit Scope Manifest contains a predeclared supplementary implementation-review override. If no Critical finding is verified and fewer than three Major findings are verified, Level Status becomes `COMPLETED` and Gate State becomes `PROCEED`. Design checks in Steps 2.5–2.8 remain reportable even when performance claims are invalidated.

Corresponding LLM Prompt:

You are performing a Level 2 (Methodological) audit of a Jupyter notebook.

You will receive the complete Audit Manifest from Level 1. Execute only IN_SCOPE steps. Preserve OUT_OF_SCOPE and NOT_APPLICABLE states, verify every applicability classification, and append a Manifest Delta without altering prior records.

Assume Level 1 (structural/reproducibility review) has already been completed. Your ONLY objective at this level is to determine whether the data pipeline and modeling methodology are scientifically sound. Do NOT comment on code style, variable naming, section headers, or deployment readiness — those belong to other audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — INFORMATION FLOW

- Locate the cell or project module where observations are assigned to training, validation, and test partitions.

- Confirm that partition assignment occurs before any fitted, target-informed, or data-adaptive transformation.

- Do not flag schema checks, deterministic parsing, immutable identifier creation, or predeclared label-independent filtering solely because they occur before the split.

- Flag only operations whose parameters, statistics, categories, features, thresholds, or sampling rules are learned from the complete dataset before partition isolation.

- Cite the stable source-cell or project-file reference for every flagged operation.

STEP 2 — PIPELINE-LEVEL LEAKAGE

- Identify specific algorithmic instances of leakage. This includes any scaler, encoder, imputer, or feature-engineering step fitted on the full dataset rather than the training split alone.

- Crucially, identify any feature selection or dimensionality reduction (e.g., PCA, SelectKBest, correlation thresholding, variance thresholding) computed on the full dataset before the split.

- Do NOT flag cross-validation or hyperparameter-tuning leakage here — that belongs to Step 5.

STEP 3 — MISSING DATA HANDLING

- Verify missing data is handled with a strategy fit only on the training data, then applied identically to the test data. - Flag any case where the missing-data strategy is computed on the full dataset or differs between splits.

STEP 4 — INGESTION CONTRACT VALIDATION - Check schema, data types, shapes, ranges, null constraints, and other predeclared invariants at load time. - Verify that exploratory or decision-guiding distribu-

tional profiling uses only the training partition. - Verify that validation and test partitions receive only predefined contract checks before final evaluation.

STEP 5 — CROSS-VALIDATION INTEGRITY

- Confirm cross-validation folds preserve the train/test separation identified in Step 1. - Check whether any fold-selection, resampling, or preprocessing step inside the CV loop exposes validation or test data to training.

STEP 6 — EVALUATION METRIC APPROPRIATENESS - Identify the evaluation metric(s) used. - State whether the metric fits the task and class distribution (e.g., flag accuracy as a red flag on an imbalanced classification problem).

STEP 7 — HYPERPARAMETER-SELECTION BOUNDARIES - Confirm selection uses training-derived resampling, cross-validation, or a designated validation partition. - Accept nested cross-validation when outer assessment folds remain isolated. - Flag any use of the final test set for tuning, feature selection, threshold selection, or model choice.

STEP 8 — BASELINE COMPARISON - Check whether model performance is compared against a meaningful baseline (naive predictor, simpler model, or published benchmark). - Flag if performance is reported in isolation with no baseline context.

OUTPUT FORMAT — return exactly these five items, followed by the machine-readable Manifest Delta, and nothing else:

1. Data pipeline integrity report (Steps 1–4): for every step emit PROVISIONAL_PASS, PROVISIONAL_FLAG, OUT_OF_SCOPE, or NOT_APPLICABLE, with evidence and stable references.

2. ML correctness checklist (Steps 5–8), using the same statuses and evidence requirements, even when performance claims are invalidated.

3. Performance-claim validity recommendation: PROVISIONALLY_VALID or INVALIDATION_RECOMMENDED, with every affected metric, comparison, selection decision, and conclusion identified.

4. Provisional aggregate Level 2 risk score: Low / Moderate / High, derived from all applicable Steps 1–8 under the severity rubric.

5. Provisional state and post-verification recommendation: - Current Level Status: PAUSED_FOR_VERIFICATION. - Current Gate State: PAUSE. - Recommend PROCEED after verification only when no release-blocking methodological finding is confirmed and fewer than three Major findings are confirmed within Level 2. - Recommend STOP after verification when leakage or another release-blocking methodological failure is confirmed. - Do not issue the final transition as an LLM decision.

Manifest Delta: append structured control outcomes, finding records, evidence, provisional severities, verification states, affected claims, Level Status, and Gate State.

Do not propose fixes or rewrite code.

Level 3 — Implementation Purpose Determine whether the applicable implementation is maintainable and release-ready after Levels 1 and 2 have completed. Level 3 runs normally only after a `PROCEED` Gate State. It may run after an earlier verified failure only as a supplementary review explicitly authorized in the Audit Scope Manifest; its results cannot change the earlier failure.

Step 3.1 — Flag repetitive code blocks: Identify code blocks that exceed the three-block threshold (three or more similar blocks) and mark them as refactoring candidates.

Step 3.2 — Identify dead code and unused elements: Flag unused imports, dead code paths, and redundant variable assignments.

Step 3.3 — Assess naming quality: Evaluate whether variable and function names are meaningful and used consistently throughout the notebook.

Step 3.4 — Check output hygiene: Flag bloated cell outputs (e.g., printing entire dataframes, raw tensors, or large arrays) that clutter the notebook without adding interpretive value.

Step 3.5 — Verify the artifact lifecycle: Verify compliance with the *Execution-Attempt and Release-Run Contract* defined in Part I. Confirm that checkpoint, exploratory, and partial outputs are stored only under Execution Attempt Identifiers in the development namespace and cannot overwrite, replace, promote, or verify a release artifact. Confirm that a release-bearing Run Identifier is used only for a fresh-kernel, end-to-end execution of the complete applicable workflow; that the run remains `COMPLETE_UNVERIFIED` until all required verification checks succeed; and that only `COMPLETE_VERIFIED` artifacts are treated as release, publication, or deployment artifacts. Re-execution shall create a new execution-attempt record and shall not silently overwrite a verified release artifact set.

Step 3.6 — Check inference/training separability: Confirm inference logic is separable from training logic — i.e., a saved model could be loaded and used for inference without re-running the training pipeline.

Step 3.7 — Check resource documentation: Verify that compute and memory constraints are documented or profiled (e.g., expected runtime, GPU/CPU requirements, peak memory usage).

Step 3.8 — Check environment completeness: Confirm a complete environment export is present and up to date (consistent with what Level 1 checked for reproducibility, but here verified for deployment sufficiency specifically).

Step 3.9 — Flag data privacy concerns: Identify any handling of PII or sensitive data without appropriate safeguards (anonymization, access control, exclusion from version control).

Level 3 deliverables: Code-quality report, deployment-readiness report, aggregate Level 3 risk score derived from all applicable Steps 3.1–3.9, Gate State transition, and manifest delta.

Deterministic Gate Decision: A provisional Critical implementation finding, a provisional aggregate *High Risk* Level 3 score, or unresolved evidence that could alter the Level 3 gate shall set Level Status to `PAUSED_FOR_VERIFICATION` and Gate State to `PAUSE`. This includes any suspected credential, PII, sensitive-data, unsafe-export, or unauthorized-disclosure condition identified in Step 3.9.

After verification, Level Status becomes `TERMINATED` and Gate State becomes `STOP` when Level 3 contains one or more verified Critical findings or three or more verified Major findings. A verified Step 3.9 exposure classified as Critical independently blocks release.

If all applicable Level 3 controls and applicability classifications are verified and Level 3 contains no release-blocking condition, Level Status becomes `COMPLETED` and Gate State becomes `PROCEED`. A *Moderate Risk* Level 3 result may proceed only to the Final Audit Gate, where it produces a conditional or limited-scope outcome as applicable; `PROCEED` is not itself a release approval.

When Level 3 is executed as a supplementary review after an earlier verified `STOP`, its own Level Status may become `COMPLETED`, but the governing Gate State remains `STOP` and its findings cannot reverse the earlier failure.

Corresponding LLM Prompt:

You are performing a Level 3 (Implementation) audit of a Jupyter notebook.

You will receive the complete Audit Manifest from Level 2. Execute only IN_SCOPE steps, preserve OUT_OF_SCOPE and NOT_APPLICABLE states, verify applicability evidence, and append a Manifest Delta.

Levels 1 and 2 have completed. Determine from the manifest whether this is a normal review after PROCEED or a supplementary review authorized after an earlier failure. Your ONLY objective at this level is to assess code maintainability and deployment readiness. Do NOT re-evaluate section structure, data leakage, model correctness, or evaluation metrics — those belong to other audit levels and are out of scope here.

Follow these steps in order:

STEP 1 — REPETITIVE CODE - Identify any code pattern that repeats across three or more cells or blocks (e.g., repeated plotting logic, repeated preprocessing steps applied separately to each feature). - Flag each as a refactoring candidate with the stable source references.

STEP 2 — DEAD CODE AND UNUSED ELEMENTS - Identify unused imports. - Identify dead code paths (code that cannot be reached or has no downstream effect). - Identify redundant variable assignments (e.g., a variable reassigned without ever being read in between).

STEP 3 — NAMING QUALITY - Assess whether variable and function names are descriptive and consistent throughout (e.g., not 'df1', 'df2', 'tmp', 'x2' used interchangeably with meaningful names elsewhere). - Flag inconsistencies where the same concept is named differently in different cells.

STEP 4 — OUTPUT HYGIENE - Identify any cell output that prints an entire dataframe, raw tensor, or large array without truncation, sampling, or summarization. - Flag these as clutter that reduces notebook readability.

STEP 5 — ARTIFACT LIFECYCLE - Apply the Execution-Attempt and Release-Run Contract defined in Part I. - Confirm that checkpoint, exploratory, and partial outputs are associated only with Execution Attempt Identifiers and remain in the development namespace. - Flag any partial output that overwrites, replaces, promotes, or verifies a release artifact as a provisional release-blocking finding. - Confirm that a release-bearing Run Identifier is used only for a fresh-kernel, end-to-end execution of the complete applicable workflow. - Confirm that the release run remains COMPLETE-UNVERIFIED until all required contracts, constraints, outputs, and integrity checks are verified, and that only COMPLETE-VERIFIED artifacts are eligible for release, publication, or deployment. - Confirm that re-execution creates a new execution-attempt record and does not silently overwrite a verified release artifact set.

STEP 6 — INFERENCE/TRAINING SEPARABILITY - Determine whether a saved model could be loaded and used for inference without re-running the training pipeline. - Flag if inference logic is entangled with training logic in the same cells with no clear separation point.

STEP 7 — RESOURCE DOCUMENTATION - Check whether the notebook documents or profiles compute/memory requirements (expected runtime, GPU/CPU needs, peak memory). - Flag if no such documentation exists, especially for computationally expensive cells (training loops, large matrix operations).

STEP 8 — ENVIRONMENT COMPLETENESS - Verify a complete, up-to-date environment export exists (e.g., requirements.txt, environment.yml) sufficient for someone else to reproduce a deployment-ready environment — not just "the code runs," but "the exact dependency versions are captured."

STEP 9 — DATA PRIVACY AND SENSITIVE INFORMATION - Identify any personally identifiable, confidential, regulated, or otherwise sensitive data loaded, displayed, logged, cached, exported, or committed by the notebook or its local project modules. - Check for anonymization or pseudonymization, access controls, retention restrictions, secret exclusion, and version-control safeguards where applicable. - Flag exposed sensitive records, embedded credentials, unrestricted exports, or missing safeguards with stable source references.

OUTPUT FORMAT — return exactly these four items, followed by the machine-readable Manifest Delta, and nothing else:

1. Code-quality report (Steps 1–4): for every step emit PROVISIONAL_PASS, PROVISIONAL_FLAG, OUT_OF_SCOPE, or NOT_APPLICABLE, with evidence and stable references.

2. Deployment-readiness report (Steps 5–9), using the same statuses and evidence requirements.

3. Provisional aggregate Level 3 risk score: Low / Moderate / High, derived from all applicable Steps 1–9. Code-quality findings shall not be excluded from this score.

4. Provisional state and deterministic post-verification recommendation: - Set the current Level Status to PAUSED_FOR_VERIFICATION and the current Gate State to PAUSE whenever a provisional Critical finding, a provisional High Risk Level 3 score, or unresolved gate-relevant evidence exists. - Treat any suspected credential, PII, sensitive-data, unsafe-export, or unauthorized-disclosure condition from Step 9 as gate-relevant and recommend PAUSE pending verification. - Recommend STOP after verification when one or more Critical findings or three or more Major findings are confirmed in Level 3. - Recommend PROCEED after verification only when every applicable control and applicability classification is verified and no release-blocking condition is confirmed. - State that PROCEED advances the audit to the Final Audit Gate and is not an independent release approval. - If Level 3 is supplementary after an earlier verified STOP, preserve the governing Gate State as STOP regardless of the Level 3 result. Manifest Delta: append structured control outcomes, finding records, evidence, provisional severities, verification states, affected artifacts or claims, Level Status, and Gate State.

Do not propose fixes or rewrite code.

Audit Provenance and Conflict Resolution

To ensure audits are actionable in production or regulatory contexts, the review process must generate a structured trail.

Audit Manifest and Inter-Level Execution Contract:

Every audit shall maintain a machine-readable and human-readable Audit Manifest bound to a definitive project snapshot. Level 1 creates the manifest; each subsequent level consumes the preceding version and appends its own results without altering prior records.

The manifest shall contain:

- `Audit ID`: unique identifier for the audit;
- `Notebook Identifier`: filename or registry identifier;
- `Notebook Source Hash`: cryptographic hash of the exact `.ipynb` file;
- `Audit Package Hash`: hash or Git commit identifier for the complete audited project surface;
- `Prompt/Protocol Version`: exact Lap and prompt revision;
- `Auditor Identity`: human reviewer, deterministic tool, or LLM configuration;
- `Stable Source Map`: notebook-cell and project-file reference map;
- `Audit Scope Manifest`: per-step scope/applicability state, evidence, exclusions, overrides, and FULL_SCOPE or LIMITED_SCOPE;
- `Parent Manifest Hash`: hash of the manifest consumed by the current level;
- `Level Status`: current execution state;
- `Control Outcome Records`: one evidence-backed outcome for every in-scope or not-applicable control;
- `Finding Records`: structured findings generated by the current level;

- `Level Risk Score`: provisional or verified aggregate risk for the level;
- `Gate State`: whether execution may proceed, must pause for verification, or must terminate;
- `Verification State`: provisional, confirmed, rejected, or unresolved; and
- `Timestamp`: UTC completion time for the current level.

Each finding record shall contain:

- a unique finding identifier;
- the originating level and step;
- a stable source reference;
- the finding statement;
- the supporting evidence;
- its provisional severity;
- its verification state;
- the final verified severity, when available; and
- any affected metric, artifact, claim, or gate.

Permitted Level Status values are:

> `NOT_STARTED`, `IN_PROGRESS`, `PAUSED_FOR_VERIFICATION`, `COMPLETED`, and `TERMINATED`.

Scope and applicability are recorded separately as `IN_SCOPE`, `OUT_OF_SCOPE`, or `NOT_APPLICABLE`. Permitted Gate State values are `PROCEED`, `PAUSE`, and `STOP`.

State transitions shall be recorded explicitly:

- provisional gate-triggering evidence: Level Status `PAUSED_FOR_VERIFICATION`, Gate State `PAUSE`;
- verified release-blocking failure: Level Status `TERMINATED`, Gate State `STOP`;
- completed level without a release-blocking failure: Level Status `COMPLETED`, Gate State `PROCEED`; and
- supplementary execution after an earlier failure: current Level Status may become `COMPLETED`, but the governing Gate State remains `STOP`.

A subsequent level shall not execute under `PAUSE`. It shall not execute under `STOP` unless a predeclared supplementary-review override applies. Every override-based execution shall preserve the failure and mark its results as supplementary.

Operational Prompt Output Contract:

Each Lap 3 prompt shall return both its specified human-readable report and a machine-readable Manifest Delta. The delta shall contain all new or updated control outcomes, finding identifiers, evidence, provisional severities, verification states, affected claims or artifacts, aggregate level risk, Level Status, Gate State, timestamp, and parent-manifest hash. The exact-output restriction of a prompt applies to these two coordinated artifacts and shall not be interpreted as permission to omit mandatory manifest fields.

Cross-Level Conflict Resolution:

The Final Audit Gate shall apply the following precedence rules:

1. **Incomplete verification:** If any `IN_SCOPE` control is unexecuted, or any `IN_SCOPE` or `NOT_APPLICABLE` outcome remains provisional, unresolved, or paused, the final status is `AUDIT INCOMPLETE: VERIFICATION PENDING`. No pass or failure release decision shall be issued.
2. **Execution termination without complete evidence:** If required clean execution could not be attempted because the audit package or environment was unavailable, the status is `AUDIT INCOMPLETE: REQUIRED EXECUTION UNAVAILABLE`. If execution was attempted and failed, its verified cause is scored normally under Level 1.
3. **Level 1 failure:** A verified High Risk Level 1 result yields `FAILED: NON-REPRODUCIBLE OR UNSAFE`. Supplementary reviews do not change this status.
4. **Level 2 data-integrity failure:** Verified pipeline leakage or another Critical methodological failure yields `FAILED: DATA INTEGRITY OR METHODOLOGICAL VIOLATION` and invalidates every affected performance claim.
5. **Other verified High Risk:** Any other verified Critical finding, three Major findings within one level, or four Major findings across the audit yields `FAILED: HIGH AGGREGATE RISK`.
6. **Limited-scope completion:** If any step is `OUT_OF_SCOPE` and no failure applies, the status is `LIMITED-SCOPE AUDIT COMPLETED: LOW RISK` or `LIMITED-SCOPE AUDIT COMPLETED: MODERATE RISK`. This status is not a release approval.
7. **Full-scope conditional pass:** If every applicable step is completed and verified, no step is `OUT_OF_SCOPE`, no failure applies, and at least one level is Moderate Risk, the status is `CONDITIONALLY PASSED: MODERATE RISK`.
8. **Full-scope aggregate pass:** The status `PASSED: LOW RISK` is permitted only when every applicable control is completed and verified, all `NOT_APPLICABLE` classifications are verified, no step is `OUT_OF_SCOPE`, every executed level is Low Risk, and no performance claim has been invalidated.

The statuses `PASSED: HIGH RISK` and unrestricted `PASSED` for a limited-scope audit are not permitted.